

Curs 1 — Heaps, Heapsort & Priority Queues (sinteză)

Relevanță la examen: acest curs acoperă **toată secțiunea §2.8 din harta de examen** (HEAPIFY + numărare swap-uri, secvențe heapable, nr. de min-heap-uri, heapsort max/min swaps, BuildHeap $O(n)$). E una dintre cele mai dese teme la grilă. Tot ce e marcat 🔥 e fix material de examen.

Două lucruri critice de la început (le explic detaliat mai jos):

- ⚠️ **Eroare în slide-uri:** deck-ul scrie că Heapify „takes linear time” — **e greșit**, Heapify e $O(\log n)$ (logaritmic). BuildHeap e cel liniar.
- ⚠️ **Capcană de notație:** slide-urile sunt **1-indexate** (rădăcina la $A[1]$). Examenul tău folosește deseori **0-indexare** (rădăcina la $A[0]$). Formulele diferă — le am pe ambele mai jos.

Întrebările „de discutat în clasă” lăsate fără răspuns în slide-uri sunt rezolvate inline (marcate ▶ **Întrebare din curs**) și recapitulate la final în §10.

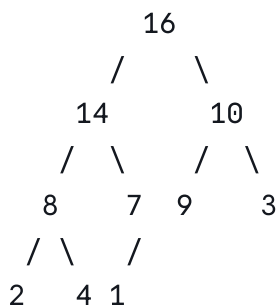
1. Ce este un heap

Un **heap** (binar) este un **arbore binar complet** (sau „aproape complet” / *nearly complete*, cum îl numește CLRS) care satisface **proprietatea de heap**.

▶ **Întrebare din curs (slide 2):** „Ce face un arbore binar să fie complet (complete)?”

Răspuns: un arbore binar e *complet/aproape complet* dacă **toate nivelurile sunt complet umplute, cu posibila excepție a ultimului nivel, care e umplut de la stânga la dreapta** (fără goluri). Cu alte cuvinte: nu ai voie să lași o poziție liberă în stânga și să pui un nod în dreapta ei.

▶ **Întrebare din curs (slide 2):** „Exemplul de mai sus e complet?” Exemplul din curs:



Răspuns: DA. Are 10 noduri. Ultimul nivel (2, 4, 1) e umplut de la stânga, fără goluri. E un arbore binar complet valid.

🔥 **Atenție conceptual:** un heap **NU** este un arbore de căutare (BST). Singura garanție e relația părinte-copil. **Între frați și între subarbori diferiți nu există nicio ordine.** (Ex: în

heap-ul de sus, $14 > 10$ nu e o regulă — putea fi invers; doar fiecare părinte $\geq$ copiii lui contează.) E o confuzie clasică.

2. Reprezentarea ca vector (array)

În practică, heap-urile se implementează ca **vectori** (nu cu pointeri), tocmai pentru că forma e completă și nu „risipești” spațiu.

Heap-ul de mai sus devine:

```
indice:  1   2   3   4  5  6  7  8  9 10
A       = [16, 14, 10,  8, 7, 9, 3, 2, 4,  1]
```

Formulele de navigare

Convenție 1-indexată (ca în slide-uri, rădăcina = $A[1]$):

Operație	Formulă
Rădăcina	$A[1]$
Părintele lui i	$A[\lfloor i/2 \rfloor]$ (împărțire întreagă)
Copilul stâng al lui i	$A[2i]$
Copilul drept al lui i	$A[2i + 1]$

⚠ **Convenție 0-indexată** (cum apare des la examenul tău, rădăcina = $A[0]$):

Operație	Formulă
Rădăcina	$A[0]$
Părintele lui i	$A[\lfloor (i-1)/2 \rfloor]$
Copilul stâng al lui i	$A[2i + 1]$
Copilul drept al lui i	$A[2i + 2]$

🔥 **De reținut:** numărul de SWAP-uri într-un exercițiu nu se schimbă în funcție de convenție — doar formulele de calcul al indicilor diferă. Dar dacă examenul îți dă un vector

și te întreabă „cine e părintele lui $A[4]$ ”, **trebuie să știi care convenție se folosește**. Verifică mereu.

► **Întrebare din curs (slide 6): „Cum implementezi asta cel mai eficient? Really?”**

Răspuns: poți folosi operații pe biți: $\text{Left}(i) = i \ll 1$, $\text{Right}(i) = (i \ll 1) \mid 1$, $\text{Parent}(i) = i \gg 1$. Dar „Really?”-ul profesorului e ironic: **orice compilator modern transformă deja $i*2$ în shift și $i/2$ în shift automat**, deci scrierea manuală cu shift-uri nu aduce niciun câștig real și doar înrăutățește lizibilitatea. E o „optimizare” inutilă (premature optimization). Concluzia didactică: scrie cod clar, lasă compilatorul să optimizeze.

3. Proprietatea de heap (max-heap vs min-heap)

Max-heap (cel din curs):

$$A[\text{Parent}(i)] \geq A[i] \quad \text{pentru orice } i > 1$$

Adică **valoarea unui nod $\leq$ valoarea părintelui său**.

► **Întrebare din curs (slide 7): „Unde se află cel mai mare element într-un heap?”**

Răspuns: în rădăcină, $A[1]$. Pentru că fiecare nod e $\leq$ părintele lui, urcând mereu ajungi la maxim în vârf.

🔥 **Min-heap** (NU e în slide-uri, dar e esențial la examen — harta lucrează cu min-heap!):

$$A[\text{Parent}(i)] \leq A[i]$$

Adică fiecare nod $\geq$ părintele $\Rightarrow$ **cel mai mic element e în rădăcină**.

Toate operațiile (Heapify, Insert, BuildHeap, Heapsort) sunt identice pentru min-heap, doar **inversezi semnul comparației** ($>$ devine $<$).

Definiții (cad la examen ca atare)

- **Înălțimea unui nod** = numărul de **muchii** de pe cel mai lung drum descendent până la o frunză.
 - **Înălțimea arborelui** = înălțimea rădăcinii.
 - (Reamintire din harta de examen: convenția uzuală e că **un nod singur are înălțime 0.**)
-

► **Întrebare din curs (slide 8):** „Care e înălțimea unui heap cu n elemente? De ce?”

Răspuns: $\Theta(\log n)$, mai exact $\lfloor \log_2 n \rfloor$. **De ce:** un arbore binar complet dublează numărul de noduri la fiecare nivel (1, 2, 4, 8, ...). Ca să încapă n noduri, ai nevoie de aproximativ $\log_2 n$ niveluri.

🔥 **Consecința cheie (motivul pentru care heap-urile sunt utile):** operațiile de bază pe heap costă cel mult proporțional cu înălțimea = $O(\log n)$. Toată valoarea structurii vine de aici.

5. Heapify() — „scufundarea” unui element (sift-down)

Heapify (numit și *sift-down* sau *max-heapify*) restabilește proprietatea de heap într-un nod.

Presupunere: subarborii din stânga și dreapta lui i sunt **deja heap-uri valide**, dar $A[i]$ poate fi prea mic $\Rightarrow$ îl lași să „coboare” până ajunge la locul lui.

► **Întrebare din curs (slide 9):** „Subarborii din i poate încălca proprietatea de heap.

Cum?” Răspuns: $A[i]$ poate fi mai mic decât unul sau ambii copii ai săi. Asta se întâmplă tipic (a) în **Heapsort**, când mutăm un element mic în rădăcină după extragerea maximumului, sau (b) în **BuildHeap**, când valoarea de la i a fost pusă arbitrar. Copiii sunt heap-uri corecte, dar vârful e „nelalocul lui”.

► **Întrebare din curs (slide 9):** „Care va fi operația de bază între i , l și r ?” **Răspuns:** **comparația** — găsești cel mai mare dintre $A[i]$, $A[l]$, $A[r]$, apoi **faci swap** între $A[i]$ și cel mai mare copil (dacă e cazul) și **continui recursiv** în jos.

Pseudocod (max-heap)

```
Heapify(A, i):
    l = Left(i);  r = Right(i)
    if l ≤ heap_size and A[l] > A[i]:
        largest = l
    else:
        largest = i
    if r ≤ heap_size and A[r] > A[largest]:
        largest = r
    if largest ≠ i:
        Swap(A[i], A[largest])
        Heapify(A, largest)    # recursie în jos
```

(Pentru **min-heap**: schimbi $\boxed{>}$ în $\boxed{<}$ și $\boxed{\text{largest}}$ → $\boxed{\text{smallest}}$.)

Exemplu complet (trace-ul din slide-uri 11-19)

Pornim de la $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ și apelăm $\text{Heapify}(A, 2)$ (valoarea 4, care e nelalocul ei):

```
PAS 1: i=2 (val 4). Copii: l=4 (val 14), r=5 (val 7).  
      Cel mai mare = 14 (indice 4). Swap  $A[2] \leftrightarrow A[4]$ .  
      → [16, 14, 10, 4, 7, 9, 3, 2, 8, 1]      recursie la i=4  
  
PAS 2: i=4 (val 4). Copii: l=8 (val 2), r=9 (val 8).  
      Cel mai mare = 8 (indice 9). Swap  $A[4] \leftrightarrow A[9]$ .  
      → [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]      recursie la i=9  
  
PAS 3: i=9 (val 4). Nu are copii (ies din heap). STOP.
```

Rezultat: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ — heap valid. Elementul 4 a „coborât” pe drumul $2 \rightarrow 4 \rightarrow 9$.

Analiza Heapify()

► **Întrebare din curs (slide 20):** „Fără apelul recursiv, cât durează Heapify()?” → $\Theta(1)$ (câteva comparații + un swap). ► „De câte ori se poate auto-apela recursiv?” → de cel mult înălțimea heap-ului = $O(\log n)$ (coboară un nivel per apel). ► „Care e timpul în cel mai rău caz pe un heap de mărime n ?” → $O(\log n)$.

Demonstrația formală (slide 21-22): Subarborii din $\boxed{l}$ sau $\boxed{r}$ au cel mult $2n/3$ noduri (cazul cel mai rău: ultimul rând umplut **pe jumătate**, ceea ce face subarborile stâng „cel mai greu”, într-un raport de $2/3$). Deci:

$$T(n) \leq T(2n/3) + \Theta(1)$$

Prin **cazul 2 din Master Theorem** (aici $\boxed{\log_{3/2} 1 = 0}$, iar $\boxed{f(n) = \Theta(1) = \Theta(n^0)}$):

$$T(n) = O(\log n)$$

⚠ **EROARE ÎN SLIDE-URI (slide 22):** deck-ul scrie după asta „Thus, Heapify() takes linear time”. E greșit și se contrazice cu propriul calcul de deasupra. $O(\log n)$ e **logaritmic**, nu liniar. **Heapify = $O(\log n)$** . (Probabil autorul a confundat cu BuildHeap, care e cel liniar — vezi §6.) Reține corect: **Heapify e logaritmic**.

Construim un heap **de jos în sus** (bottom-up), apelând `Heapify()` pe noduri în ordine inversă.

► **Întrebare din curs (slide 23):** „Fapt: elementele $A[\lfloor n/2 \rfloor + 1 .. n]$ sunt deja heap-uri. De ce?” **Răspuns:** pentru că acelea sunt exact frunzele. Un nod i e frunză $\Leftrightarrow 2i > n$ (nu are copil stâng), adică $i > n/2$. **Frunzele nu au copii $\Rightarrow$ satisfac trivial proprietatea de heap** (sunt heap-uri de un singur element). Deci nu trebuie să le procesezi — pornești doar de la ultimul **nod intern**, indicele $\lfloor n/2 \rfloor$.

De ce mergem de la $\lfloor n/2 \rfloor$ în jos spre 1: ordinea inversă garantează că, **când procesezi nodul i , copiii lui sunt deja heap-uri** (au fost procesați înainte). Asta e fix presupunerea de care are nevoie Heapify.

Pseudocod

```
BuildHeap(A):  
    heap_size = length(A)  
    for i =  $\lfloor \text{length}(A)/2 \rfloor$  downto 1:  
        Heapify(A, i)
```

Exemplu complet (din slide 25)

$A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, $n=10$. Pornim de la $i=5$:

```
i=5 (val 16): copii 7.  $16 > 7$ , OK. Niciun swap.  
i=4 (val 2): copii 14, 8. Swap cu 14  $\rightarrow [4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$   
i=3 (val 3): copii 9, 10. Swap cu 10  $\rightarrow [4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$   
i=2 (val 1): copii 14, 16. Swap cu 16  $\rightarrow [4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$   
    recursie i=5 (val 1): copil 7  $\rightarrow$  swap  $\rightarrow [4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$   
i=1 (val 4): copii 16, 10. Swap cu 16  $\rightarrow [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$   
    recursie i=2 (val 4): copii 14, 7  $\rightarrow$  swap cu 14  $\rightarrow$   
     $[16, 14, 10, 4, 7, \dots]$   
    recursie i=4 (val 4): copii 2, 8  $\rightarrow$  swap cu 8  $\rightarrow$   
     $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ 
```

Rezultat final: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ — exact heap-ul canonic din §1.
(Frumos test de consistență: același vector.)

Analiza BuildHeap() — 🔥 🔥 PUNCT CLASIC DE EXAMEN

► **Întrebare din curs (slide 26):** „Este $O(n \log n)$ o margine superioară corectă?” $\rightarrow$ DA, e corectă ($n/2$ apeluri $\times O(\log n)$). E o margine validă, dar **nu strânsă (not tight)**. ► „Există o

margină mai bună?" → DA: $O(n)$. ▶ „Cum e posibil? Unde e greșeala în raționamentul $O(n \log n)$?"

Răspunsul complet (asta trebuie să știi de explicat): Greșeala e că raționamentul $O(n \log n)$ presupune că fiecare apel `Heapify` costă $O(\log n)$, dar majoritatea nodurilor sunt aproape de fund și au înălțime mică. `Heapify` pe un nod de înălțime h costă doar $O(h)$, iar:

█ **Fapt (slide 27):** un heap cu n elemente are cel mult $\lceil n/2^{(h+1)} \rceil$ noduri de înălțime h .

(Adică: $\sim n/2$ frunze de înălțime 0, $\sim n/4$ noduri de înălțime 1, $\sim n/8$ de înălțime 2 ... și un singur nod de înălțime $\log n$.) Însușind costul real:

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n \cdot 2) = O(n)$$

pentru că seria $\sum h/2^h$ converge la 2.

🔥 **Formularea scurtă de reținut:** „*BuildHeap* e $O(n)$, nu $O(n \log n)$, pentru că marea majoritate a nodurilor au înălțime mică — costul e dominat de cele multe noduri ieftine de jos, nu de cele puține scumpe de sus.” Asta poate fi exact o întrebare de Nivel II.

7. Heapsort

Având `BuildHeap`, sortarea **in-place** vine natural:

1. Maximul e la `A[1]`.
2. Îl „elimini” făcând **swap cu ultimul element** `A[n]` $\Rightarrow$ acum `A[n]` are valoarea corectă finală (cel mai mare).
3. **Decrementezi** `heap_size` (scoți ultimul din heap).
4. **Heapify(A, 1)** restabilește proprietatea pe heap-ul micșorat.
5. Repeti, mereu swap `A[1]` $\leftrightarrow$ `A[heap_size]`.

Pseudocod

```
Heapsort(A):
    BuildHeap(A)
    for i = length(A) downto 2:
        Swap(A[1], A[i])
        heap_size -= 1
        Heapify(A, 1)
```

Ideea: un max-heap scoate maximele în ordine descrescătoare și le „depune” de la coada vectorului spre cap $\Rightarrow$ vectorul rămâne **sortat crescător**.

Analiza (slide 30)

- BuildHeap: $O(n)$
- $n-1$ apeluri Heapify, fiecare $O(\log n) \Rightarrow O(n \log n)$
- Total: $O(n) + O(n \log n) = O(n \log n)$

🔥 Proprietăți de reținut:

- **In-place** ($O(1)$ memorie suplimentară).
- **Nu e stabil** (poate schimba ordinea elementelor egale).
- **$O(n \log n)$ garantat în worst-case** (spre deosebire de Quicksort, care e $O(n^2)$ în worst-case dar de obicei mai rapid în practică — de aceea Quicksort câștigă uzual, cum zice slide 31).

🔥 **Legătura cu harta (§2.8 — întrebarea ambiguă cu swap-uri):** întrebarea „heapsort: max/min swap-uri când secvența e crescătoare/descrescătoare” se simulează exact cu mecanismul de aici (BuildHeap + extrageri). Reține raționamentul: o secvență **descrescătoare** e deja un max-heap valid $\Rightarrow$ 0 swap-uri în BuildHeap. **Dar atenție: am marcat în hartă că două chei oficiale se contrazic pe acest punct — e o întrebare prost calibrată, nu pierde timp pe ea.**

8. Priority Queues (cozi de prioritate)

Heapsort e drăguț, dar adevărata valoare a heap-ului e că **implementează eficient o coadă de prioritate**.

O **coadă de prioritate** menține o mulțime (S) de elemente, fiecare cu o **cheie** (key/prioritate). Operații (pentru **max-priority-queue**):

- **Maximum(S)** — returnează elementul cu cheia maximă.
- **ExtractMax(S)** — scoate și returnează elementul cu cheia maximă.
- **Insert(S, x)** — adaugă x în S .

► **Întrebare din curs (slide 31):** „La ce ar fi utilă o coadă de prioritate?” Răspuns (exemple sugestive):

- **Planificarea proceselor** într-un sistem de operare (rulează procesul cu prioritatea cea mai mare).
- **Simulări bazate pe evenimente** (extragi mereu evenimentul cu cel mai mic timp).

- **Algoritmul lui Dijkstra** (drumuri minime) și **Prim** (MST) — extrag mereu minimul (min-priority-queue). 🔥 (Apropo de harta de examen: Prim/Dijkstra folosesc heap-uri — leagă-le.)
- **Codarea Huffman** (extragi cele două frecvențe minime).
- **A*** și alte căutări euristice.

► **Întrebare din curs (slide 32):** „Cum implementezi aceste operații cu un heap?”

Răspuns:

Maximum(S): returnezi $A[1]$. $\rightarrow O(1)$

```
Maximum(A):
    return A[1]
```

ExtractMax(S): salvezi maximul, muți ultimul element în rădăcină, micșorezi heap-ul, refaci proprietatea cu Heapify. $\rightarrow O(\log n)$

```
ExtractMax(A):
    max = A[1]
    A[1] = A[heap_size]
    heap_size -= 1
    Heapify(A, 1)
    return max
```

Insert(S, x): pui x pe ultima poziție, apoi îl „ridici” (sift-up / bubble-up) cât timp e mai mare decât părintele. $\rightarrow O(\log n)$

```
HeapInsert(A, key):
    heap_size += 1
    i = heap_size
    A[i] = key
    while i > 1 and A[Parent(i)] < A[i]:      # min-heap: schimbă < în >
        Swap(A[i], A[Parent(i)])
        i = Parent(i)
```

9. 🔥 Sift-UP vs Sift-DOWN — clarificarea care îți rezolvă întrebarea cu swap-uri din hartă

Asta e confuzia #1 a studenților și e **fix mecanismul din întrebarea de examen** „inserează 5,4,3,2,1 într-un min-heap, câte swap-uri?”.

	Sift-DOWN (Heapify)	Sift-UP (la Insert)
Direcția	de sus în jos	de jos în sus
Când	ExtractMax, BuildHeap	Insert (adăugare element)
Comparați cu	copiii	părintele
Acțiune	cobori dacă ești mai mic (max-heap)	urci dacă ești mai mic (min-heap)

🔥 **Întrebarea din hartă** („inserează 5,4,3,2,1 în min-heap adăugând la coadă + HEAPIFY, câte SWAP-uri?") **este de fapt sift-UP (Insert)**, nu sift-down. Numeri swap-urile **cumulat** la fiecare inserare:

```

Insert 5: [5]                → 0 swap
Insert 4: 4<5, urcă          → [4,5]                → 1 swap
Insert 3: 3<4, urcă          → [3,5,4]            → 1 swap
Insert 2: 2<5, 2<3, urcă×2 → [2,3,4,5]          → 2 swap
Insert 1: 1<3, 1<2, urcă×2 → [1,2,4,5,3]        → 2 swap
                                TOTAL → 6 swap

```

Răspuns: 6. (Numărul e același indiferent de 0- sau 1-indexare.)

Bonus — „secvențe heapable" (tot din hartă): o secvență e *heapable* dacă, adăugând fiecare element la coadă, el e **deja** $\geq$ **părintele său** (min-heap) $\Rightarrow$ **nu e nevoie de niciun sift-up**. Ex: $\boxed{2\ 4\ 3\ 5}$ e heapable (fiecare nou $\geq$ părinte); $\boxed{2\ 4\ 5\ 3}$ nu ($3 <$ părintele 4).

10. 🌟 Recapitulare: toate întrebările „de discutat în clasă" + răspunsuri

#	Întrebarea din slide-uri	Răspuns scurt
1	Ce face un arbore binar <i>complete</i> ?	Toate nivelurile pline, ultimul umplut de la stânga, fără goluri.
2	Exemplul e complet?	Da (ultimul nivel 2,4,1 e umplut de la stânga).
3	Cum implementezi navigarea cel mai eficient? <i>Really?</i>	Cu shift-uri pe biți — dar compilatorul o face deja, e optimizare inutilă.
4	Unde e cel mai mare element?	În rădăcină, A[1] (max-heap).

#	Întrebarea din slide-uri	Răspuns scurt
5	Înălțimea unui heap cu n elemente? De ce?	$\Theta(\log n) = \lfloor \log_2 n \rfloor$, fiindcă fiecare nivel dublează nodurile.
6	Cum poate subarborele din i să încalce heap-ul?	$A[i]$ e mai mic decât un copil (după swap în Heapsort / valoare pusă arbitrar în BuildHeap).
7	Care e operația de bază între i, l, r?	Comparația (găsești maximul) + swap + recursie.
8	Fără recursie, cât durează Heapify?	$\Theta(1)$.
9	De câte ori se auto-apelează?	De cel mult înălțimea = $O(\log n)$.
10	Worst-case Heapify pe n elemente?	$O(\log n)$. ⚠️ (NU „linear”, cum greșește slide-ul!)
11	De ce $A[\lfloor n/2 \rfloor + 1..n]$ sunt heap-uri?	Sunt frunzele — nu au copii, deci satisfac trivial heap-ul.
12	E $O(n \log n)$ corectă pentru BuildHeap? E strânsă?	Corectă, dar NU strânsă.
13	De ce e de fapt $O(n)$? Unde e greșeala?	Majoritatea nodurilor au înălțime mică; $\sum (n/2^{(h+1)}) \cdot h = O(n)$ (seria converge la 2).
14	La ce e utilă o coadă de prioritate?	Scheduling, simulări, Dijkstra/Prim, Huffman, A^* .
15	Cum implementezi Insert/Maximum/ExtractMax cu heap?	Maximum = $A[1]$ $O(1)$; ExtractMax = mută ultimul în rădăcină + Heapify $O(\log n)$; Insert = pune la coadă + sift-up $O(\log n)$.

11. ⚡ Checklist examen pentru acest curs

Trebuie să poți face fără ezitare:

- ☐ Să desenezi heap-ul ↔ vectorul în ambele sensuri (1- și 0-indexat).
- ☐ Să faci un **trace de Heapify** (sift-down) pas cu pas.
- ☐ Să faci un **trace de BuildHeap** (de la $\lfloor n/2 \rfloor$ în jos).
- ☐ Să **numeri swap-urile la Insert** (sift-up) cumulativ — întrebarea 5,4,3,2,1 = 6. 🔥
- ☐ Să recunoști o **secvență heapable**. 🔥
- ☐ Să simulezi Heapsort (BuildHeap + extrageri).

De știut pe de rost:

- ☐ Heapify = $O(\log n)$ (NU liniar — eroarea din slide). ⚠️
- ☐ BuildHeap = $O(n)$ + **de ce** (argumentul cu înălțimi mici). 🔥 🔥
- ☐ Heapsort = $O(n \log n)$, in-place, instabil.
- ☐ Înălțimea heap-ului = $\lfloor \log_2 n \rfloor$.
- ☐ Max-heap: max în rădăcină / Min-heap: min în rădăcină.
- ☐ Heap $\neq$ BST (fără ordine între frați).
- ☐ Părinte/copii: 1-indexat $\lfloor i/2 \rfloor, 2i, 2i+1$ | 0-indexat $\lfloor (i-1)/2 \rfloor, 2i+1, 2i+2$. ⚠️
- ☐ Operații coadă de prioritate: Maximum $O(1)$, Insert & ExtractMax $O(\log n)$.